

Software Requirements Specification (SRS)

Project Name: Student Management Console Application

Phase: Week 1 Practice Project

Week 1 Learning Goals

This project directly supports the Week 1 roadmap checklist. It is designed to be **simple** (no databases, files, collections, interfaces, exceptions, or advanced Java features yet). The goal is to practice:

- Classes and Objects
- Constructors and Constructor Overloading
- Method Overloading and Overriding
- `this` and `static` keywords
- Access Modifiers
- Basic Abstraction and Polymorphism

1. Objective

Create a Java console application that manages student records in memory. The application demonstrates core Object-Oriented Programming (OOP) concepts including class/object creation, encapsulation, inheritance, polymorphism, and the use of static members.

2. Actors

- **User:** The person operating the console application.

3. Functional Requirements (FR)

FR-1: Add Student

The user can create a new student record.

- **Required Data:** Student ID, Student Name, Age, Department.
- **Example Output:**

```
ID: 101
Name: Dhanush
Age: 20
Department: CSE
```

FR-2: View Student

Display the complete information of a specific student.

- **Example Output:**

Student ID : 101
Name : Dhanush
Age : 20
Department : CSE

FR-3: Update Department

Change a student's department.

- **Example Output:**

Current Department: CSE
New Department: AI & DS

FR-4: Display Student Count

Show the total number of students created. This **must** use the `static` keyword.

- **Example Output:**

Total Students: 5

FR-5: Show Student Details

Each student object should provide a `displayDetails()` method to print their information.

4. Non-Functional Requirements (NFR)

- **NFR-1:** Console-based interface only.
- **NFR-2:** No database integration.
- **NFR-3:** No file handling/I/O operations.
- **NFR-4:** Data may be hardcoded in memory for testing purposes.
 - *Example:* `Student s1 = new Student(...);`

5. Required Class Design

Class 1: `Person` (Parent Class)

- **Attributes:**
 - `private String name;`
 - `private int age;`
- **Constructors:**
 - `Person()`
 - `Person(String name, int age)`
- **Methods:**
 - `public void displayDetails()`

Class 2: `Student` (Child Class)

- **Inherits:** `Person`
- **Additional Attributes:**
 - `private int studentId;`
 - `private String department;`
- **Constructors:**

`Student()` // Default

`Student(int id, String name, int age, String department)` // Parameterized

- **Methods:**

`@Override`

`public void displayDetails()`

6. Implementation Requirements

6.1 Constructor Overloading

The `Student` class must include at least the following constructors:

`Student()`

`Student(int id)`

`Student(int id, String name, int age, String department)`

6.2 Method Overloading

Create multiple versions of the department update method:

`public void updateDepartment(String dept)`

`public void updateDepartment(String dept, boolean printMessage)`

6.3 Static Members

Maintain the total student count across all instances.

- **Variable:** `private static int studentCount;`
- **Method:** `public static int getStudentCount()`

6.4 Encapsulation

All attributes must be `private`. Access and modification must only occur through standard `getters` and `setters`.

6.5 Polymorphism

Demonstrate dynamic method dispatch by creating a parent reference for a child object:

`Person p = new Student(...);`

`p.displayDetails();` // Output must come from the Student class implementation

7. User Interface (Console Menu)

The application must loop through a menu using a `switch` statement until the user chooses to exit.

```
===== STUDENT MANAGEMENT SYSTEM =====
```

1. Add Student
2. View Student
3. Update Department
4. Show Student Count
5. Exit

Choose an option:

8. Success Criteria Checklist

You have successfully completed this phase if your project contains:

- ☐ 1 Parent Class
- ☐ 1 Child Class
- ☐ Encapsulation (Private variables + Getters/Setters)
- ☐ Constructor Overloading
- ☐ Method Overloading
- ☐ Method Overriding
- ☐ Inheritance
- ☐ Polymorphism (Upcasting)
- ☐ Static Variable
- ☐ Static Method
- ☐ Proper Access Modifiers
- ☐ Interactive Console Menu